

```

In [1]: import os
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader

save_dir = './checkpoints'
os.makedirs(save_dir, exist_ok=True)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

BATCH_SIZE = 512
LEARNING_RATE = 0.4
MOMENTUM = 0.9
WEIGHT_DECAY = 0.0001
EPOCHS = 50

class SimCLRTransform:
    def __init__(self):
        color_jitter = transforms.ColorJitter(0.4, 0.4, 0.4, 0.1)
        self.train_transform = transforms.Compose([
            transforms.RandomResizedCrop(size=32, scale=(0.2, 1.0)),
            transforms.RandomHorizontalFlip(),
            transforms.RandomApply([color_jitter], p=0.8),
            transforms.RandomGrayscale(p=0.2),
            transforms.ToTensor(),
            transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994,
        ])

    def __call__(self, x):
        return self.train_transform(x), self.train_transform(x)

trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=
train_loader = DataLoader(trainset, batch_size=BATCH_SIZE, shuffle=True, num

class SimCLR_ResNet18(nn.Module):
    def __init__(self):
        super(SimCLR_ResNet18, self).__init__()
        base_model = torchvision.models.resnet18(weights=None)
        base_model.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=
        base_model.maxpool = nn.Identity()
        self.encoder = nn.Sequential(*list(base_model.children())[:-1])
        self.projector = nn.Sequential(
            nn.Linear(512, 512, bias=False),
            nn.Linear(512, 128, bias=False)
        )

    def forward(self, x):
        h = self.encoder(x).view(x.size(0), -1)
        z = self.projector(h)
        return h, z

```

```

model = SimCLR_ResNet18().to(device)

class NTXentLoss(nn.Module):
    def __init__(self, temperature=0.1):
        super(NTXentLoss, self).__init__()
        self.temperature = temperature
        self.criterion = nn.CrossEntropyLoss(reduction="sum")
    def forward(self, z_i, z_j):
        batch_size = z_i.size(0)
        z_i, z_j = F.normalize(z_i, dim=1), F.normalize(z_j, dim=1)
        representations = torch.cat([z_i, z_j], dim=0)
        similarity_matrix = torch.matmul(representations, representations.T)
        labels = torch.cat([torch.arange(batch_size) + batch_size, torch.arange(batch_size)], dim=0)
        mask = torch.eye(2 * batch_size, dtype=torch.bool).to(device)
        similarity_matrix.masked_fill_(mask, -9e15)
        logits = similarity_matrix / self.temperature
        return self.criterion(logits, labels) / (2 * batch_size)

criterion = NTXentLoss()
optimizer = optim.SGD(model.parameters(), lr=LEARNING_RATE, momentum=MOMENTUM)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=EPOCHS)

for epoch in range(1, EPOCHS + 1):
    model.train(); running_loss = 0.0
    for (view1, view2), _ in train_loader:
        view1, view2 = view1.to(device), view2.to(device)
        optimizer.zero_grad()
        _, z_i = model(view1); _, z_j = model(view2)
        loss = criterion(z_i, z_j); loss.backward(); optimizer.step()
        running_loss += loss.item()
    scheduler.step()
    print(f"Epoch [{epoch}/{EPOCHS}] Loss: {running_loss / len(train_loader)}")
    if epoch % 10 == 0:
        torch.save(model.state_dict(), os.path.join(save_dir, f'simclr_resnet18_epoch{epoch}.pt'))
    print("Training complete.")

```

Using device: cuda

100%|██████████| 170M/170M [00:02<00:00, 70.1MB/s]

Epoch [1/50] Loss: 6.6429
Epoch [2/50] Loss: 6.4516
Epoch [3/50] Loss: 6.3668
Epoch [4/50] Loss: 6.2877
Epoch [5/50] Loss: 5.9982
Epoch [6/50] Loss: 5.7036
Epoch [7/50] Loss: 5.6163
Epoch [8/50] Loss: 5.5367
Epoch [9/50] Loss: 5.4764
Epoch [10/50] Loss: 5.4211
Epoch [11/50] Loss: 5.3838
Epoch [12/50] Loss: 5.3414
Epoch [13/50] Loss: 5.3001
Epoch [14/50] Loss: 4.9659
Epoch [15/50] Loss: 4.8095
Epoch [16/50] Loss: 4.7347
Epoch [17/50] Loss: 4.6723
Epoch [18/50] Loss: 4.5284
Epoch [19/50] Loss: 4.2313
Epoch [20/50] Loss: 4.1400
Epoch [21/50] Loss: 4.0918
Epoch [22/50] Loss: 4.0597
Epoch [23/50] Loss: 4.0170
Epoch [24/50] Loss: 3.9887
Epoch [25/50] Loss: 3.9537
Epoch [26/50] Loss: 3.9517
Epoch [27/50] Loss: 3.9076
Epoch [28/50] Loss: 3.8869
Epoch [29/50] Loss: 3.8716
Epoch [30/50] Loss: 3.8522
Epoch [31/50] Loss: 3.8300
Epoch [32/50] Loss: 3.8220
Epoch [33/50] Loss: 3.8018
Epoch [34/50] Loss: 3.7676
Epoch [35/50] Loss: 3.7553
Epoch [36/50] Loss: 3.7412
Epoch [37/50] Loss: 3.7104
Epoch [38/50] Loss: 3.7111
Epoch [39/50] Loss: 3.6894
Epoch [40/50] Loss: 3.6734
Epoch [41/50] Loss: 3.6690
Epoch [42/50] Loss: 3.6402
Epoch [43/50] Loss: 3.6373
Epoch [44/50] Loss: 3.6351
Epoch [45/50] Loss: 3.6228
Epoch [46/50] Loss: 3.6054
Epoch [47/50] Loss: 3.6073
Epoch [48/50] Loss: 3.5947
Epoch [49/50] Loss: 3.6050
Epoch [50/50] Loss: 3.6025
Training complete.